Compact proofs on max-of-2: what an agent loop finds

Agentic loop WFJKK/Auto-Research-Compact-Proof, 13 October 2026. Two runs of Claude Fable 5.1 (effort high) through Claude Code, identical except for the metric the agent is told to maximise. All proofs checked by the exact Python checker (no Lean proofs yet).

Setup

- **Networks.** max-of-2 from the proof-based tutorial: two tokens in, the larger one out, three linear maps. v64: vocabulary 64, d_model 8, 4,096 inputs. v128: vocabulary 128, d_model 16, 16,384 inputs. Ten trained networks per size: six score the rounds, four are held out for the final evaluation.
- **Proof.** A split tree over the inputs, each piece checked by one rule (single input, interval bound over a box of inputs, or skip), with global rules for contraction, symmetry and approximation. The checker rebuilds the network from its weights and recomputes every bound in exact rational arithmetic.
- **Length** L: the exact operations the checker performs, everything included (reading weights and proof file). **Accuracy**: certified inputs over all inputs. B is the length of brute force: v64 2,696,387, v128 40,407,555.
- **Agent loop.** Each round the agent sees the frontier, its best recipes, its last attempts with diagnostics, and returns one recipe: a program from weights to proof, run at five knob values (0 to 1) that trade length against accuracy. Ten rounds per run; recipes run in a container sandbox.

Two metrics

Q averages, over budgets b spaced evenly on a log scale from E (reading the weights) to B, the best accuracy a(b) certified within b:

$$Q = \int a(b)\, d(\ln b) \,/\, \ln(B/E)$$

Cost of finishing prices every correct input a proof leaves uncertified at the brute-force cost of one input, $\lambda = B/n$, and takes the cheapest total over all proofs:

$$\text{cost of finishing} = B \,/\, \min\,(\, L + \lambda \cdot U \,)$$

with U the uncertified correct inputs. It measures complete certification only, so the best value is B over the shortest full-accuracy proof. Q also rewards partial proofs, including where on the length axis they are placed; the hand-written proof given the same five-budget knob is the fair Q baseline.

Results

Shortest full-accuracy proof (median)	v64	vs Louis	v128	vs Louis
Hand-written proof (Louis)	312,264		2,446,083	
Fable 5.1 high, scored on Q	295,698	−5.3%	2,112,262	−13.6%
same, on held-out networks	292,558	−6.3%	2,093,111	−14.4%
Fable 5.1 high, scored on cost of finishing	294,854	−5.6%	2,117,809	−13.4%
same, on held-out networks	292,558	−6.3%	2,117,356	−13.4%

Median Q	v64	v128	Cost of finishing	v64	v128
Louis, one length	0.276	0.307	Louis	8.63	16.52
Louis with a budget knob	0.323	0.347	scored on Q	9.12	19.13
scored on Q	0.393	0.404	scored on cost of finishing	9.14	19.08
scored on cost of finishing	0.325	0.348			

Full accuracy means every input the network classifies correctly (99.90 to 99.98%); every run certifies exactly that on every network, development and held out. Medians over six development networks unless marked held out (four networks).

The idea the agents found

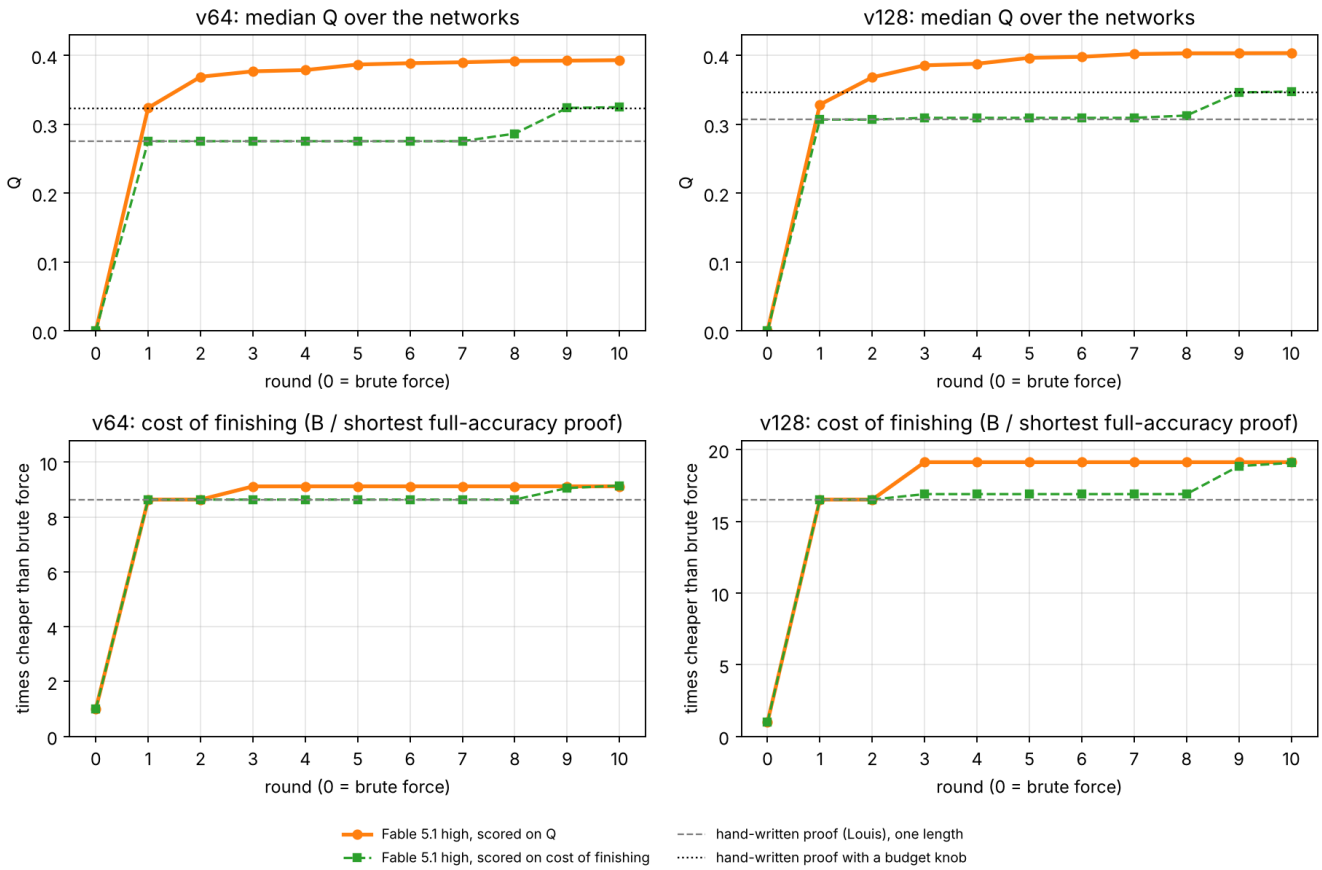
The hand-written proof contracts the three maps into one matrix M, groups inputs by the larger token b, and checks each group's smaller token a exactly, at a cost of two operations per token per output. The agents found that the margin is small only near the diagonal $a \approx b$ and grows with the gap $b - a$ (measured on the weights: median margin about 3.5 next to the diagonal, about 17 far from it). So the far region is certified through coarse piecewise-constant approximations of M's rows, whose error the checker bounds exactly and which cost one merge step per segment instead of per token, and only a narrow band near the diagonal is checked exactly. Several approximations at different error levels let each label use the coarsest one its margins allow.

- Found by both runs independently: in round 3 of the run scored on Q, and in rounds 9 and 10 of the run scored on cost of finishing, after five rounds on a rotated-basis box bound worth 2%.
- The gain is 5 to 6% on v64 and 13 to 14% on v128 on the development networks, and 6% and 14% on the held-out ones, so the recipe is not tuned to particular weights. It grows with the vocabulary, as the share of inputs far from the diagonal does.
- No run went beyond it. One agent proposed a new rule (bounding far competitors through a telescoping chain of adjacent label differences); a float simulation on the weights shows its bound passes for almost no label, so it would not help.

Reading the comparison

- **Q against cost of finishing.** The run scored on Q found the idea in round 3; the run scored on cost of finishing took nine. Q rewards cheap partial proofs, which forces the agent to sort inputs by difficulty, and that sorting is the idea. One run per metric, so this is suggestive, not established.
- **Q includes placement.** The Q run sits 0.06 to 0.07 above the knob baseline; the full-proof gain accounts for about 0.02 to 0.03 of that, the rest is better partial proofs and where they are placed. A fixed budget grid would remove the placement part.

Caveats: one run per metric; Python-checked only.



Top: median Q per round. Bottom: cost of finishing per round, B over the shortest full-accuracy proof so far. Each round is one Fable response; round 0 is brute force.